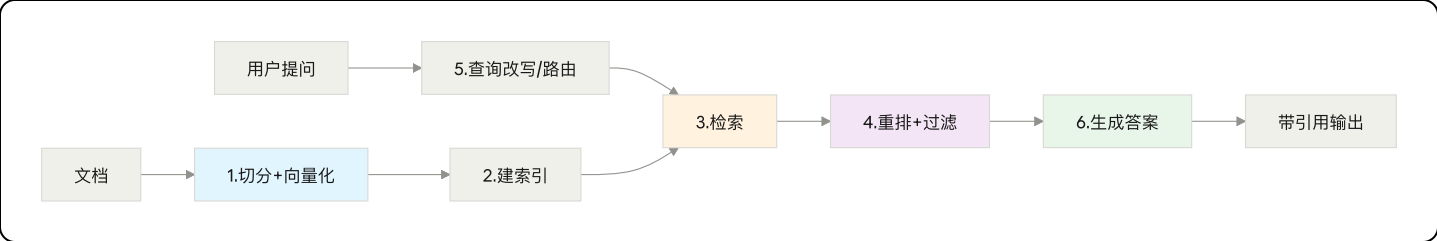


RAG 高级技术笔记（2026）

一句话理解 RAG：给 LLM 一场"开卷考试"——先从外部知识库中找到相关资料，再让模型基于资料作答，从而压制幻觉、补充时效与私域知识。

本笔记按 RAG 的数据主线组织，每个技术都明确归位到它解决的是哪一环：



1. 切分与向量化（Chunking & Embedding）

为什么要切分

一段聚焦单一主题的短文本，其向量语义远比"一整页大杂烩"精准。检索的本质是比相似度，块越聚焦，命中越准。

🍰 比喻：把书拆成便利贴，每张只记一件事。贴得越专一，找起来越快。

块大小的权衡

没有万能值，核心是两头取舍：

- 块太小 → 语义破碎、丢上下文
- 块太大 → 向量被稀释、检索不精准、还浪费 LLM 上下文

主流切分策略（按精细度递进）

策略	做法	适用
固定窗口	按固定字数/token 切，带重叠	简单、基线方案
结构感知	按标题、段落、Markdown/代码结构切	文档有清晰层级时
语义分块	按语义边界切，避免把一句话拦腰斩断	追求块内主题一致
晚分块（Late Chunking）	先对长文整体编码，再切分，让每块"记得"全文上下文	上下文连贯性要求高

向量化

选嵌入模型可参考 **MTEB** 等公开评测，但务必结合自己领域的数据小规模实测——榜单存在刷榜现象，且通用最优 ≠ 你的场景最优。

💡 选型还要看：是否多语言、是否支持长文本、是否支持指令/查询-文档非对称编码、推理成本。

2. 建立搜索索引 (Indexing)

向量存好后，需要一个能快速"找最近邻"的结构。

方案	原理	适用
Flat (暴力检索)	逐个算距离	数据量小、要求绝对精确
ANN 近似检索 (HNSW / IVF 等)	用图、聚类、树近似找邻居	数据量大、可接受微小精度损失
向量数据库 (Pinecone / Weaviate / Qdrant / Milvus 等)	封装存储+检索+元数据过滤+数据摄入	生产环境，免造轮子

💡 同时存**元数据** (日期、来源、权限等)，检索时用过滤器精准缩小范围，也是实现权限隔离的关键。

2.1 层次索引 (应对海量文档)

建**两层**：一层文档**摘要**，一层文档**块**。检索分两步——先用摘要筛出相关文档，再只在这些文档内细查。

📖 比喻：先翻**目录**定位章节，再翻到具体页看细节，而不是一页页硬找。

2.2 上下文增强 (检索小块，喂给 LLM 大块)

核心矛盾：小块检索更准，大块上下文更全。两种调和法：

方法	做法	一句话理解
句子窗口检索	按单句嵌入；命中后把该句 前后各 k 句 一起返回	找到关键句，顺手带上"左邻右舍"
父子块 (自动合并)	检索小子块；若前 k 个结果中有多个指向 同一父块 ，就用整个父块替换	碎片凑够了，干脆"合并成整块"

🧩 **比喻**：想引用某句话，光一句说不清——把它所在的**整段**一起拿出来佐证。

2.3 假设性问题 / HyDE（可选增强）

两种"以问钓答"的反向技巧：

- **假设性问题**：让 LLM 给每个块预生成"它能回答的问题"，索引这些问题；用户提问时做问题对问题匹配，语义更接近。
- **HyDE**：让 LLM 先根据提问编一个假想答案，用假答案的向量去检索。

🎣 **比喻**：HyDE 像甩"假鱼饵"下水——假答案和真文档长得像，更容易钓到相关内容。💎 **取舍**：长上下文与强基座模型普及后，这类技巧收益下降，主要在小模型、垂直领域库中仍有价值。属可选项而非必备。

3. 混合检索（Hybrid Search）

把两条检索路融合，已是 2026 年的默认配置：

- **关键词检索**（稀疏，如 BM25）：擅长精确术语、专有名词、代号、罕见词
- **语义检索**（稠密向量）：擅长理解意图、近义改写、模糊表达

⚖️ 关键在于合理融合两套不同量纲的评分（常用 RRF，倒数排名融合），融合后几乎总要再接一道重排。🆕 **进阶**：多向量 / 后期交互检索（如 ColBERT 类）在词级别做细粒度匹配，精度更高、成本也更高，适合对召回质量极敏感的场景。

4. 重排与过滤（Rerank & Filter）

检索回来的候选良莠不齐，这是喂给 LLM 前的最后一道质检：

- **过滤**：按相似度阈值、关键词、元数据（如时间新近性、来源可信度）筛掉杂质。
- **重排**：用更强的模型（交叉编码器 Cross-Encoder、Cohere/Jina 等 Rerank 服务、或 LLM 直接打分）重新精排。

👤 **比喻**：检索是"海投后初筛简历"，重排是"叫来候选人面试精排"——更慢更准，所以只对少量候选做。

5. 查询转换与路由（Query Transformation & Routing）

在检索之前，先用 LLM 加工用户的问题。

5.1 查询改写与分解

- **改写/扩展**：把口语化、含糊的提问改写得更适合检索（补全省略、加同义词）。
- **分解**：复杂问题拆成可独立检索的子问题，并行检索后汇总。

例：「Langchain 和 LlamaIndex 谁的 GitHub star 更多？」→ 拆成「Langchain 多少 star？」 + 「LlamaIndex 多少 star？」→ 分别查 → 合并作答。

5.2 查询路由

LLM 做的**决策**：这个问题该走哪条路？

- 直接回答 / 查某个特定索引 / 走多条路再综合
- 选择数据源：向量库、图数据库、关系库、Web 搜索，或某一层索引

 **比喻**：像分诊台护士，先判断病人该去哪个科室，而不是所有人排同一个队。

6. 对话引擎（多轮上下文）

要支持追问、指代（"它""这个"）、上下文相关指令，关键是结合聊天历史压缩查询：

模式	做法
拼接上下文	先检索相关内容，连同聊天历史一起发给 LLM（简单直接）
压缩再检索	先把"历史 + 最新消息"压缩成一个独立查询，再去检索（更精确，避免历史污染检索）

 **比喻**：你问"它多少钱？"，引擎得先回看上一句聊的是哪件商品，才能把"它"翻译成具体查询。

7. 生成答案：响应合成（Response Synthesis）

最后一步：基于检索到的全部上下文 + 原始问题，让 LLM 产出答案。

- **最简单**：所有相关上下文 + 问题一次性拼进 prompt（上下文不长时首选）。
- **应对超长上下文**：
 1. **迭代精炼**：分批喂上下文，逐步打磨答案。
 2. **先总结**：把上下文压缩到能塞进 prompt。
 3. **并行+合并**：对不同上下文块各生成答案，再拼接/总结。

参考引用 (Citation)

与其说是检索技巧，不如说是**可信度工具**——但极其重要，是 RAG 能落地于严肃场景的前提。

当答案来自多个来源时，要能**溯源**：

1. 在 prompt 中要求 LLM 标注所用来源的 ID。
2. 把生成内容与原始块做**模糊匹配**回溯来源。

 引用还是对抗幻觉的抓手：让答案的每句话都能指回原文，便于人工核查与自动校验。

8. 智能体化 RAG (Agentic RAG)

让 LLM **自主决策检索流程**，而非固定的“一次检索一次回答”。典型能力：判断**要不要检索**、**检索几次**、**检索什么**、**结果不满意是否重新检索**。

多文档智能体方案：给每份文档配一个 agent（能摘要、能问答，手握向量索引+摘要索引两件工具）；上层一个**顶级 agent** 负责把问题分派给各文档 agent 并综合答案。

 **比喻**：顶级 agent 是项目经理，每个文档 agent 是负责一份资料的专员；经理派活、专员各自查、最后汇总。**优势**：能跨文档比较不同方案/实体，兼顾单文档摘要与问答，覆盖大多数文档交互场景。 这一范式在 2026 年已从“高级玩法”走向主流——配合工具调用，RAG 越来越像一个会自己查资料的智能体，而非静态管线。

9. GraphRAG (图增强检索)

用**知识图谱**组织知识：从文档中抽取实体与关系建图，检索时既能查局部细节，也能沿关系做**全局性、多跳推理**。

 **擅长的问题**：「总结整个知识库的核心主题」「A 和 C 通过哪些中间实体关联」——这类需要跨多个文档、多个实体串联的问题，纯向量检索往往答不好。 **代价**：建图与维护成本高，适合知识结构化、关系密集领域（如医疗、法律、企业知识库）。

10. 评估与可观测性 (不可跳过)

RAG 不是搭完就完事，必须**能量化好坏**，否则无法迭代。

两层指标：

- **检索质量**：召回率、命中率、上下文相关性——找回来的料对不对？

- **生成质量**：忠实度（答案是否严格基于上下文，不编造）、答案相关性——答得对不对、靠不靠谱？

📌 常用"LLM 作为裁判（LLM-as-judge）"自动化打分，配合人工抽检。**先建评估集，再调系统**——没有度量就没有优化方向。

11. 安全与生产化要点

- **权限与隔离**：检索必须尊重用户权限，按元数据过滤，避免越权泄露私域数据。
- **抗注入**：被检索到的文档可能含恶意指令（间接 prompt 注入），需对检索内容做隔离与防护，不能无条件信任。
- **成本与延迟**：重排、多跳、Agentic 流程都增加调用次数；按场景权衡精度与开销，能用缓存就缓存。
- **新鲜度**：建立增量更新机制，让索引跟上数据变化。

12. 长上下文 vs RAG：还需要 RAG 吗？

百万级 token 模型出现后常有此问。结论倾向**互补而非替代**：

维度	长上下文直接塞	RAG
知识时效	受训练截止限制	可随时更新
成本	长输入越来越贵	只取相关片段，更省
可溯源	难精确引用	天然带来源
私域/超大库	塞不下	按需检索

长上下文削弱了"纯粹为塞进 token 上限而切块"的动机，但**时效、成本、溯源、规模**这几点决定了 RAG 依然不可替代。二者常组合使用。

📌 一页速记

环节	核心技术	一句话
切分	固定 / 结构 / 语义 / 晚分块	块要"恰好聚焦一件事"

环节	核心技术	一句话
向量化	选模型 + 领域自测	别盲信榜单
索引	Flat / ANN / 向量库 + 层次索引	先查目录再翻页
增检索	句子窗口 / 父子块 / HyDE	找得准 + 看得全
检索	混合检索 + RRF	关键词+语义双引擎
重排	Cross-Encoder / Rerank	初筛后再面试
查询	改写 / 分解 / 路由	拆问题、选路线
对话	历史压缩查询	听懂"它"指什么
生成	迭代 / 总结 / 并行 + 引用	整合并溯源
智能体	Agentic / 多文档 agent	自主决定怎么查
图谱	GraphRAG	多跳与全局推理
工程	评估 / 权限 / 抗注入 / 成本	度量驱动、安全优先